

Frameworks

TCP2201

Object-oriented Analysis and Design



Lecture outcomes

- At the end of this lecture, you should
 - learn about object-oriented frameworks
 - distinguish between frameworks and design patterns
 - identify when to use frameworks



What are frameworks?

- A framework is a reusable, “*semi-complete*” application that can be specialized to produce custom applications
- They are usually designed specifically to solve unique domain problems (e.g. best UI practices, real-time processing etc.) and/or target specific business units (e.g. fast data processing etc.)
- A framework implements a generic solution to a generalized problem to allow it to provide common facilities applicable to different application programs.
 - Principle: Applications that do different, but related, things tend to have quite similar designs
- Examples of frameworks include Microsoft’s MFC and Javasoft’s RMI.



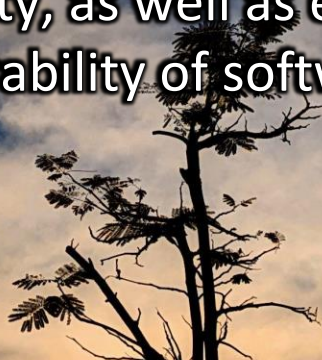
OO frameworks

- In the object-oriented paradigm, a framework is composed of a library of classes
 - The API is defined by the set of all public methods of these classes.
 - Some of the classes will be abstract and there are often many interfaces
- Examples
 - A framework for payroll management
 - A framework for frequent buyer clubs
 - A framework for university registration
 - A framework for e-commerce web sites



Benefits of OO frameworks

- Modularity – Frameworks enhance modularity by encapsulating volatile implementation details behind stable interfaces → helps improve software quality by localizing the impact of changes. This localization reduces the effort required to understand and maintain existing software.
- Reusability – The interfaces provided by frameworks enhance reusability by defining generic components that can be reapplied to create new applications. Framework reusability avoids re-creating and re-validating common solutions to recurring application requirements and software design challenges.
Reuse of framework components can yield substantial improvements in programmer productivity, as well as enhance the quality, performance, reliability and interoperability of software.



- Extensibility – A framework enhances extensibility by providing explicit hook methods that allow applications to extend its stable interfaces without affecting existing services. Framework extensibility is essential to ensure timely customization of new application services and features.
- Inversion of control – The run-time architecture of a framework is characterized by an “inversion of control” This architecture enables repeated application processing steps to be customized by event handler objects that are invoked via the framework's reactive dispatching mechanism. Inversion of control allows the framework (rather than each application) to determine which set of application-specific methods to invoke in response to external events (such as window messages arriving from end-users or packets arriving on communication ports).



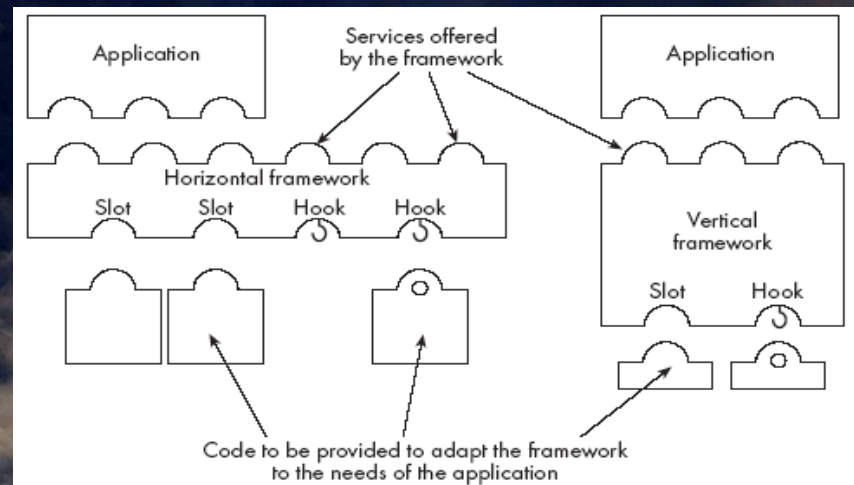
Frameworks promote reuse

- A framework is intrinsically (and intentionally) incomplete
 - Certain classes or methods are used by the framework, but are missing (slots)
 - Some functionality is optional
 - Allowance is made for developer to provide it (hooks or extension points)
 - Developers use the services that the framework provides
 - Taken together the services are called the Application Program Interface (API)



Types of frameworks

- A horizontal framework provides general application facilities that a large number of applications can use
- A vertical framework (application framework) is more 'complete' but still needs some slots to be filled to adapt it to specific application needs



- Frameworks can also be classified by the techniques used to extend them.
 - Whitebox frameworks (providing hook methods)
 - Blackbox frameworks (specifying call back methods)

Using a framework

- How to use a framework:
 - Hook methods provide access to framework
 - Can be used or overridden by an application
 - Call back methods
 - Must be defined by applications
 - Are called in response to events recognized by the framework



Examples of frameworks (DB)

- Medoo - <http://medoo.in/>



- Laminas - <https://getlaminas.org/>

La///inas

Examples of frameworks (Gaming)

- Unity - <https://unity.com/>



- Unreal - <https://www.unrealengine.com>



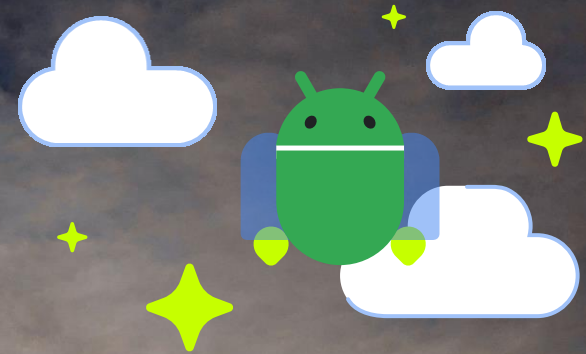
UNREAL
TECHNOLOGY

Examples of frameworks (mobile)

- Apple iOS - <https://developer.apple.com/>



- Android Jetpack - <https://developer.android.com/jetpack>



- Cross-platform Flutter - <https://docs.flutter.dev/>



Flutter

Comparing DP and frameworks

- DPs and frameworks are very highly related, and neither is subordinate of the other
- Patterns are typically more abstract/general descriptions of frameworks
- Frameworks implement patterns usually in a particular language
- Design patterns are the architectural building blocks of frameworks. They help make frameworks extendable and reusable. Frameworks usually contain implementations of many cooperating design patterns



Summary

- Design patterns are partial solutions to common problems such as separating an interface from a number of alternate implementations, wrapping around a set of legacy classes and protecting a caller from changes associated with specific platforms.
- DPs assist in creating better software design with methods that have been tried and tested

